

Guion de presentación — Defensa PFG: JerezSur Inmobiliaria

Tiempo total estimado: 15-20 min de slides + demo

ANTES DE EMPEZAR

- Abre el panel de administración en local y ten iniciada la sesión.
 - Ten abierta la pestaña del feed XML (</api/portal/feed.xml>) lista para mostrar.
 - Comprueba que el frontend público carga con inmuebles visibles.
 - Volumen del micro apagado o bajado.
-

SLIDE 1 — PORTADA (~30 seg)

"Buenos días. Soy David Orellana Gómez, y voy a presentarles JerezSur Inmobiliaria: una plataforma digital completa, desarrollada a medida, para el sector inmobiliario local."

"El proyecto nace de una necesidad real: digitalizar la operativa de una agencia familiar de Jerez de la Frontera que gestionaba su día a día con herramientas de terceros que no se adaptaban a su flujo de trabajo."

SLIDE 2 — CONTEXTO, MOTIVACIÓN Y SOSTENIBILIDAD (~1:30 min)

"El sector inmobiliario local arrastra dos problemas importantes."

"Primero: las webs de las agencias pequeñas son obsoletas, sin diseño adaptable, y no transmiten confianza al cliente."

"Segundo: dependen de CRMs externos como Inmoweb, con cuotas SaaS mensuales, interfaces sobrecargadas y, lo más crítico, sin soberanía sobre sus propios datos."

"La solución que planteo es un portal y un panel de gestión propios, contruidos a medida del flujo de trabajo real de esta agencia. El dato es de la agencia, no de ningún intermediario."

"En cuanto a sostenibilidad: el proyecto se alinea con el ODS 9 —industria e innovación— y aplica principios de Green Code: paginación en todas las consultas para no cargar miles de registros, imágenes servidas por CDN con compresión automática, y un microservicio Python que genera gráficos solo cuando se solicitan."

SLIDE 3 — PLANIFICACIÓN, METODOLOGÍA Y VIABILIDAD (~1:30 min)

"La metodología seguida es un ciclo de vida incremental e iterativo inspirado en Scrum, con tres sprints bien diferenciados."

"El Sprint 1 sentó la infraestructura: Docker, base de datos, seguridad y modelos." "El Sprint 2 construyó las interfaces y la conectividad frontend-backend." "El Sprint 3 integró los módulos restantes — sindicación XML, análisis Python, notificaciones— y cerró el proyecto."

"La gestión de tareas y requisitos se hizo con GitHub Projects, y el control de versiones siguió el flujo GitHub Flow: rama main siempre estable, desarrollo en ramas feature."

"En cuanto a viabilidad económica: el presupuesto de desarrollo asciende a 4.620 euros para el primer año, con un coste recurrente de solo 135 euros en VPS y dominio. Entra perfectamente dentro de la línea del Kit Digital, que cubre hasta 6.000 euros, con ROI positivo desde el primer año al eliminar las licencias SaaS de terceros."

SLIDE 4 — ARQUITECTURA DEL SISTEMA (~2:00 min)

"La arquitectura es desacoplada y se compone de cuatro servicios orquestados con Docker Compose."

"El frontend —tanto el panel admin como el portal público— son SPAs de React que se comunican con el backend mediante REST y JSON sobre HTTPS."

"El backend es una API Spring Boot que estructura su lógica en tres capas: controladores, que solo reciben peticiones y delegan; servicios, donde vive toda la lógica de negocio; y repositorios, interfaces JPA que Spring traduce a SQL automáticamente."

"La capa de persistencia es MySQL, con un volumen Docker para que los datos sobrevivan reinicios del contenedor."

"Y hay un cuarto servicio que aparecerá más adelante: un microservicio Python/Flask para la generación de gráficos del dashboard."

"Las variables sensibles —contraseña de base de datos, secreto JWT, credenciales de Cloudinary— no están en el código: se inyectan como variables de entorno en tiempo de ejecución, por lo que no aparecen en el repositorio Git."

SLIDE 5 — EL CORAZÓN: BACKEND Y DATOS (~2:00 min)

"El esquema relacional en MySQL está normalizado y optimizado. Todos los scripts DDL están versionados en el repositorio."

"Quiero detenerme en un hito técnico que marcó el proyecto: el incidente INC-001, que llamé internamente 'De Contrato a Operación'."

"El diseño inicial vinculaba el Contrato directamente al Inmueble. Al implementar los casos de uso reales de la agencia, detecté que un proceso de compraventa puede tener primero un contrato de arras y luego uno de compraventa definitivo, sobre la misma venta. Con el modelo original, eso era imposible."

"La solución fue introducir una entidad intermedia: Operacion. Un inmueble puede tener varias operaciones, y cada operación puede tener varios contratos en distintos estados —borrador, pendiente de firma, firmado, cancelado—."

"Además, Operacion es una clase abstracta con herencia SINGLE_TABLE: OperacionVenta y OperacionAlquiler se guardan en la misma tabla con una columna discriminadora. Esto permite consultas eficientes sin UNION."

"Esta refactorización costó ocho horas, pero salvó la integridad del negocio."

SLIDE 6 — LÓGICA DE SERVIDOR Y SEGURIDAD (~ 1:30 min)

"La seguridad se implementa con Spring Security y JWT."

"El flujo es: el usuario hace login con email y contraseña; Spring Security verifica la contraseña con BCrypt; el servidor genera un token JWT firmado con HS256; el cliente lo guarda y lo adjunta en la cabecera Authorization de cada petición."

"Un filtro JwtFilter intercepta todas las peticiones, valida el token y carga el usuario en el contexto de seguridad. Si el token es inválido o ha expirado, la petición se rechaza con 401."

"BCrypt es importante: añade una sal aleatoria por cada hash, por lo que dos hashes del mismo texto son siempre distintos. Esto protege ante ataques de diccionario y tablas rainbow."

"En cuanto a RGPD: las contraseñas se almacenan siempre hasheadas, nunca en texto plano. Los usuarios no se borran físicamente —soft delete—, lo que permite mantener el historial de citas y contratos vinculados a ellos sin violar la integridad referencial. Y en el primer login de un usuario creado por el admin, el sistema le obliga a cambiar la contraseña temporal generada por OTP."

SLIDE 7 — INTERFAZ DE USUARIO Y CLIENTE (~ 1:30 min)

"El frontend está construido en React con TypeScript y se diseñó con enfoque Mobile-First."

"Los estilos están escritos íntegramente en SASS con una arquitectura de ficheros modular: variables globales, componentes reutilizables y estilos por página separados. No se usa ninguna librería de componentes externa; todo el sistema de diseño —botones, formularios, tarjetas, cabecera— está implementado a medida con BEM como convención de nomenclatura."

"El patrón arquitectónico principal son los custom hooks: cada entidad —inmuebles, citas, trabajadores, contratos— tiene su hook propio que encapsula el estado, las llamadas a la API con Axios y el manejo de errores. El componente de página solo consume el hook y renderiza."

"Esto tiene una ventaja clara: si cambia un endpoint de la API, solo se modifica el hook correspondiente, no todos los componentes que lo usan."

"Dos hooks utilitarios reseñables: useBodyScroll, que bloquea el scroll del body mientras hay un modal abierto para evitar el doble scroll; y useFormSubmit, que encapsula el patrón try-catch-loading de cualquier formulario."

"La desviación de quince horas respecto a la planificación inicial fue asumida conscientemente para dominar React en profundidad y garantizar código limpio y mantenible."

SLIDE 8 — ANÁLISIS DE DATOS (~ 1:00 min)

"El dashboard de administración incluye gráficos estadísticos generados por un microservicio Python independiente."

"¿Por qué Python y no Chart.js directamente en el frontend? Tres razones: primero, la calidad visual de matplotlib es el estándar en el ámbito científico y genera imágenes exportables. Segundo, la lógica de datos queda en el servidor, no expuesta al cliente. Y tercero, era el módulo natural para integrar la asignatura de análisis de datos."

"El flujo es: Spring Boot recoge los datos de la BD, los envía al microservicio Flask en formato JSON, Python genera el gráfico con matplotlib y lo devuelve como imagen PNG codificada en base64. El frontend lo muestra directamente en una etiqueta img."

"Se generan dos gráficos: uno de barras agrupadas con operaciones mensuales por tipo —ventas y alquileres—, y una dona con la distribución de clientes por perfil."

"El servicio Python corre en el puerto 5000 dentro de la red interna de Docker: no está expuesto al exterior."

SLIDE 9 — EL FACTOR INNOVADOR: SINDICACIÓN AUTOMÁTICA (~2:00 min)

"Este es el módulo diferenciador del proyecto, y me gustaría dedicarle un momento."

"Las agencias inmobiliarias publican sus inmuebles en Idealista, Fotocasa y otros portales de forma manual: acceden a cada portal, rellenan los formularios, suben las fotos. Si tienen veinte inmuebles y cinco portales, son cien operaciones redundantes cada vez que hay un cambio."

"Este módulo elimina ese problema. El sistema genera automáticamente un feed XML dinámico bajo el estándar que Idealista y Fotocasa esperan de sus agencias partners."

"Cuando el agente crea o edita un inmueble en el panel, el feed se actualiza instantáneamente. Los portales apuntan a la URL de nuestro servidor —GET /api/portal/feed.xml— y recogen el inventario completo con un único crawl periódico."

"Técnicamente: el feed se construye con la API DOM estándar de Java, sin librerías externas. Cada nodo incluye identificación, datos comerciales, localización —con provincia inferida del código postal—, características técnicas, gastos como comunidad e IBI, documentación como el certificado energético, e imágenes ordenadas con la portada en primera posición porque los portales la usan como thumbnail."

"Hay tres endpoints: el feed completo, uno filtrado por ventas, y otro por alquileres, para agencias que operan en plataformas especializadas."

"Resultado: noventa por ciento menos de duplicidad de tareas. Una agencia familiar compite en visibilidad con grandes promotoras desde un único punto de gestión."

SLIDE 10 — CONTROL DE CALIDAD Y EVALUACIÓN (~1:00 min)

"En cuanto a calidad: el backend cuenta con pruebas unitarias en JUnit para los servicios principales, y la integración de todos los endpoints fue verificada con Postman."

"El análisis estático con SonarLint durante el desarrollo reportó cero code smells críticos ni blockers al cierre del sprint final."

"El score de Lighthouse del frontend público fue del 62% en el entorno local. Quiero ser transparente sobre este dato: el 62% corresponde exclusivamente a la métrica de Performance, medida con tres contenedores Docker corriendo simultáneamente en la misma máquina —MySQL, Spring Boot, React dev server y el microservicio Python— compitiendo por CPU y RAM. Los scores de Accessibility y Best Practices superan el 90%."

"En un VPS con procesamiento aislado y Cloudinary gestionando las imágenes con compresión automática, el objetivo es alcanzar un 85 o superior. Es una mejora pendiente, no un defecto de diseño."

SLIDE 11 — DEMO EN VIVO (~4:00 min, ajustable)

"Voy a hacer un recorrido por el ciclo de vida completo de un inmueble."

[Paso 1 — Login]

"Accedemos al panel de administración. El login es seguro: credenciales verificadas con BCrypt, token JWT generado y almacenado en el cliente."

[Paso 2 — Alta de inmueble]

"Creamos un nuevo inmueble. Rellenamos los campos, subimos imágenes —que van directamente a Cloudinary, el CDN externo—, marcamos cuál es la portada." "Observad que el formulario tiene un selector con búsqueda en tiempo real para asignar propietarios con su porcentaje de titularidad. La suma debe sumar cien por ciento."

[Paso 3 — Listado público]

"En el portal público, el inmueble ya aparece en el catálogo. Los filtros de búsqueda funcionan en tiempo real: tipo, precio, habitaciones, operación."

[Paso 4 — Feed XML]

"Y aquí el factor diferenciador: el feed XML. Accedemos a /api/portal/feed.xml y vemos el inventario completo ya incluye el inmueble que acabamos de crear, listo para ser consumido por Idealista o Fotocasa."

SLIDE 12 — CONCLUSIONES Y TRABAJO FUTURO (~1:00 min)

"Para cerrar: el proyecto cumple todos los requisitos funcionales y no funcionales planteados en el anteproyecto, con módulos adicionales no previstos inicialmente —la sindicación XML y el microservicio Python— que aportan valor real al negocio."

"El stack elegido —Spring Boot, React, MySQL, Docker— es profesional, estándar en la industria, y la arquitectura desacoplada facilita el mantenimiento y la evolución del sistema."

"Como líneas futuras: revisión con el cliente real y ajustes de UX basados en feedback, integración de pasarelas de firma electrónica para los contratos, y la posibilidad de comercializar la plataforma como SaaS para otras agencias de la provincia."

"Muchas gracias por su atención. Quedo a disposición del tribunal para las preguntas."

TURNOS DE PREGUNTAS — QUICK REFERENCE

Si preguntan por...	Ir a sección del cheatsheet
JWT / sesiones	\$3
BCrypt / contraseñas	\$3
Relaciones JPA / circulares	\$4, \$5
DTOs	\$7
Transacciones / rollback	\$6
Soft delete / RGPD	\$9
XML / sindicación	\$19
Operación vs Contrato	\$20
Python / gráficos	\$21
Lighthouse 62%	\$22
JWT en localStorage	\$22
SINGLE_TABLE inheritance	\$20
Paginación	\$16
Custom hooks React	\$11, \$16
Docker / despliegue	\$15
CORS	glosario \$18

CONTROL DE TIEMPO

Slides	Tiempo acumulado
1-3 (intro + contexto + planificación)	~3:30
4-6 (arquitectura + backend + seguridad)	~7:00
7-8 (frontend + Python)	~9:30
9-10 (sindicación + calidad)	~12:30
Demo	~16:30

Slides**Tiempo acumulado**

Conclusiones

~17:30

Si el tribunal pide reducir tiempo: comprime slides 3 y 8 (30 seg cada una), y la demo a 2 minutos (login + feed XML solamente).